Assignment 1

Monte Carlo Methods for Financial Applications

5MA178

Markus Ådahl

Sam Moeini and André Johansson
samo0171, anyp0002

Due Date: May 14, 2024

Contents

1 Introduction

In this assignment we will price different European options by modeling the stock price $S(t)$ that follows a geometric Brownian motion. We will conduct 20,000 simulations for some option pricing calculation and vary the number of simulations from 10,000 to 100,000 in other cases to see how the results change and present this comparison using line plots. All calculations and has been done through python.

2 Task 1: Call and put option

2.1 Method/pseudocode for Task 1

Algorithm 1 Option Pricing using Black-Scholes Formula

procedure BLACKSCHOLES(S_0, r, σ, T, K)
    Calculate d_1 and d_2:
    $d_1 = \frac{\log(S_0/K)+(r+0.5\cdot\sigma^2)\cdot T}{\sigma\cdot\sqrt{T}}$
    $d_2 = d_1 - \sigma \cdot \sqrt{T}$
    Calculate call and put prices:
    $C = S_0 \cdot \text{norm.cdf}(d_1) - K \cdot e^{-r\cdot T} \cdot \text{norm.cdf}(d_2)$
    $P = putcallparity$
    return C, P
end procedure

Algorithm 2 Study Option Price depending on volatility(σ), Time to maturity(T) and interest rate(r)

for each σ in range [0.01, 0.5] **do**
    Calculate call and put prices using Black-Scholes
end for
for each T in range [1/12, 5] **do**
    Calculate call and put prices using Black-Scholes
end for
for each r in range [0, 0.1] **do**
    Calculate call and put prices using Black-Scholes
end for

Algorithm 3 Monte Carlo Simulation for Option Pricing

procedure MonteCarlo(S_0, r, σ, T, K, n)
    Generate n random paths for stock prices using GBM formula:
    $S_T = S_0 \cdot e^{(r-0.5\cdot\sigma^2)\cdot T+\sigma\cdot\sqrt{T}\cdot Z}$
    Calculate payoffs for call and put:
    call_payoff $= \max(S_T - K, 0)$
    put_payoff $= \max(K - S_T, 0)$
    Discount payoffs to present value and compute average:
    average_call $= \frac{1}{n}\sum \mathrm{e}^{-rT} \cdot$ call_payoff
    average_put $= \frac{1}{n}\sum \mathrm{e}^{-rT} \cdot$ put_payoff
    Calculate 95% confidence interval for both call and put prices:
    CI_call $=$ average_call $\pm\, 1.96 \cdot \frac{\sigma_{\text{call}}}{\sqrt{n}}$
    CI_put $=$ average_put $\pm\, 1.96 \cdot \frac{\sigma_{\text{put}}}{\sqrt{n}}$
    return average call and put prices and their confidence intervals
end procedure

Algorithm 4 Monte Carlo Simulation using Antithetic Variables

procedure MonteCarloAntithetic(S_0, r, σ, T, K, n)
    Initialize array for antithetic stock prices
    for $i = 1$ to n **do**
        Generate Z_i and $-Z_i$
        Calculate $S_T^+ = S_0 \cdot e^{(r-0.5\cdot\sigma^2)\cdot T+\sigma\cdot\sqrt{T}\cdot Z_i}$
        Calculate $S_T^- = S_0 \cdot e^{(r-0.5\cdot\sigma^2)\cdot T+\sigma\cdot\sqrt{T}\cdot(-Z_i)}$
        Store both S_T^+ and S_T^-
    end for
    Calculate payoffs for call and put using both S_T^+ and S_T^-
    Average payoffs from S_T^+ and S_T^-
    Discount averaged payoffs to present value and compute average
    Calculate 95% confidence interval using standard deviation of payoffs
    return average prices and confidence intervals
end procedure

2.2 Table with results from 1a, 1c and 1d

The following tables presents the calculated prices for call and put options using different methods, including exact values, standard Monte Carlo (SMC), and antithetic variates, with their corresponding 95% confidence intervals for various strike prices:

Table 1: Comparison of Call Option Pricing Methods

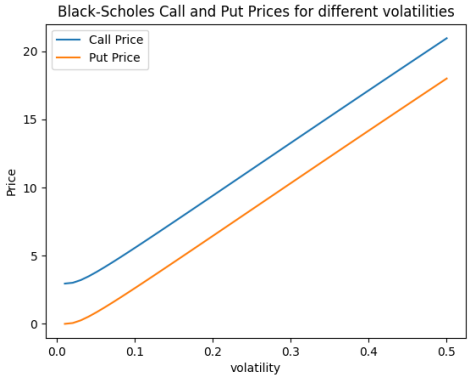
Method	K=70	K=100	K=130
Exact	32.235	9.413	1.358
Standard MC	[32.027, 32.302, 32.578]	[9.268, 9.464, 9.659]	[1.274, 1.352, 1.431]
Antithetic	[31.931, 32.205, 32.479]	[9.299, 9.496, 9.693]	[1.281, 1.357, 1.433]

Table 2: Comparison of Put Option Pricing Methods

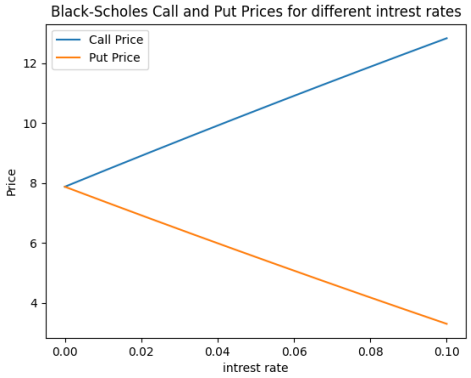
Method	K=70	K=100	K=130
Exact	0.166	6.458	27.516
Standard MC	[0.139, 0.154, 0.170]	[6.296, 6.426, 6.555]	[27.388, 27.629, 27.869]
Antithetic	[0.146, 0.162, 0.178]	[6.384, 6.514, 6.644]	[27.259, 27.501, 27.743]

2.3 1b: Call and put price with strike price 100 with depending variables: Volatility, time to maturity, and the interest rate

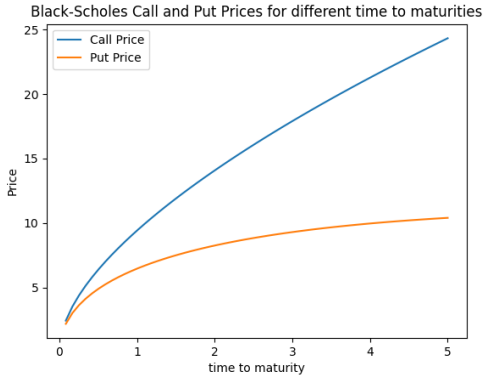
The following figures present a comparison between how the call and put prices depend on different variables: Volatility, interest rate, and time to maturity.



(a) Varying Volatility



(b) Varying Interest Rate



(c) Varying Time to Maturity

Figure 1: Comparison of Call and Put Prices Depending on Different Variables

2.4 1e: Comparison of Option Pricing Methods

The following figure presents a comparison between the Standard Monte Carlo (SMC) method, Antithetic Variates (AV), and the analytical Black-Scholes model for pricing a call option with a strike price of 100. The comparison is shown as a function of the number of simulations used:

Figure 2: Comparison plot between SMC price, AV price, and Black Scholes model.

2.5 Discussion: Task 1

Provide intuitive (not based on formulas) explanations for the plots in b) for volatility and time to maturity.

Volatility plot
As volatility increases the price of the call and put option also increases. This happens because higher volatility increases the potential for the underlying asset's price to rise above/under the strike price by the option's expiration making the option more valuable.

Time to Maturity plot
Call Price: The price of the call option increases as the time to maturity increases. A longer duration until expiration gives the underlying asset more time to potentially exceed the strike price thereby increasing the value of the call option.

Put Price: The put price increases but at a slower rate compared to the call price as time to maturity increases. While more time also means more chance for the asset to decline below the strike price the effect is generally less deciding than for call options. This is due to how gains and losses are perceived and priced. The asymmetry arises because over time asset increase is seen as more likely due to the market behavior and economic growth behavior. Mathematically we can see that as T (Time to maturity) increases the put option price decreases while the call option price increases in black scholes formula:

$$C = S_0\Phi(d_1) - Ke^{-rT}\Phi(d_2) \tag{1}$$

$$P = Ke^{-rT}\Phi(-d_2) - S_0\Phi(-d_1) \tag{2}$$

3 Task 2: Asian Call

3.1 Method/pseudocode for Task 2

Algorithm 5 Pricing Asian Options with Geometric and Arithmetic Means

procedure PRICEASIANOPTIONS($s0$, r, σ, T, m, K)
    Initialize time steps ti based on m
    Calculate $dt = \frac{T}{m}$
    Initialize stock prices S with $S[0] = s0$
    for $i = 1$ to m **do**
        $S[i] = S[i-1] \cdot \exp((r - 0.5 \cdot \sigma^2) \cdot dt + \sigma \cdot \sqrt{dt} \cdot Z_i)$
    end for
    Calculate geometric mean $G = \left(\prod_{i=1}^{m} S[i]\right)^{1/m}$
    Calculate arithmetic mean $A = \frac{1}{m}\sum_{i=1}^{m} S[i]$
    Calculate payoffs:
    Geometric payoff $= \max(G - K, 0)$
    Arithmetic payoff $= \max(A - K, 0)$
    Discount payoffs to present value:
    $PV_{geo} = \mathrm{e}^{-rT} \cdot$ Geometric payoff
    $PV_{arith} = \mathrm{e}^{-rT} \cdot$ Arithmetic payoff
    Calculate control variate adjusted price:
    $b = \frac{\mathrm{cov}(PV_{geo}, PV_{arith})}{\mathrm{var}(PV_{geo})}$
    $CV_{price} = PV_{arith} - b \cdot (PV_{geo} - \text{Expected Geo})$
    return PV_{geo}, PV_{arith}, CV_{price}
end procedure

3.2 Results: Task 2

Table 3: The following table presents the calculated prices for options using different methods

Option - Method	**K = 70**		**K = 100**		**K = 130**	
	m=12	m=52	m=12	m=52	m=12	m=52
GAC – Exact	30.381	30.283	5.435	5.167	0.106	0.076
AAC – Standard MC	30.558	30.655	5.596	5.314	0.122	0.100
AAC – CV	30.706	30.611	5.630	5.359	0.128	0.098

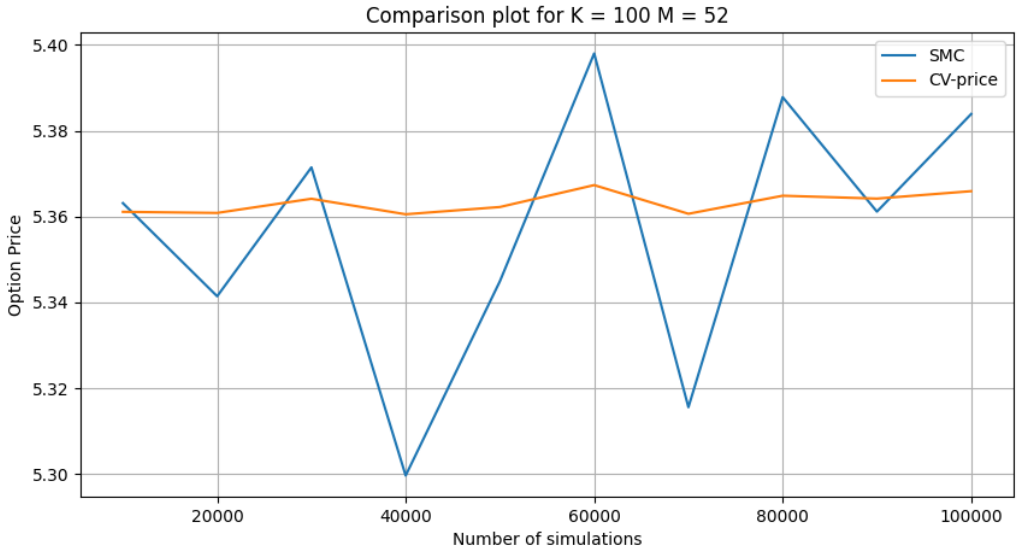


Figure 3: Comparison plot between SMC-price and CV-price for 10 000 - 100 000 simulations

3.3 Discussion: Task 2

Why is the price of the geometric option lower than the arithmetic option?
The price of the geometric option is lower than the arithmetic option due to differences in their averaging methods and volatility sensitivity. Geometric options use the geometric mean which tends to be lower than the arithmetic mean because it includes extreme values and is less sensitive to high volatility. This results in geometric options having lower volatility and risk which leads to a lower option price compared to arithmetic option

Why does the price decrease as m increases?
As the number of time steps m increases each time increment $\Delta t = \frac{T}{m}$ (T = time to maturity) becomes smaller leading to more frequent but smaller adjustments in the simulated stock price paths. This results in a reduction of the volatility impact per step as the term $\sigma\sqrt{\Delta t}$ decreases with smaller Δt. Therefore extreme price fluctuations are

less occurring and both the geometric and arithmetic averages of the stock prices become less volatile and converge towards more central values. This lowers the estimated option price because the payoff calculations includes less extreme values on the calculated option value.

4 Task 3: Barrier option, Down-and-In-Call

4.1 Method/ pseudocode for Task 3

Algorithm 6 Monte Carlo Simulation for Down-and-In Call Option Pricing

procedure MonteCarloDownAndIn(S_0, r, T, K, b, m, σ)
    Initialize $dt = \frac{T}{m}$
    Initialize $payoff$ array of zeros for each path
    Generate paths for the underlying asset S
    for each path **do**
        $S[0] = S_0$
        for $t = 1$ to m **do**
            Generate Z, a standard normal variable
            $S[t] = S[t-1] \cdot \exp((r - 0.5 \cdot \sigma^2) \cdot dt + \sigma \cdot \sqrt{dt} \cdot Z)$
            if $S[t] \leq b$ **then**
                Record the minimum time t_i such that $S[t_i] \leq b$
            end if
        end for
        if $\min_{t_i} S[t_i] \leq b$ **then**
            $payoff = \max(S[m] - K, 0)$
        else
            $payoff = 0$
        end if
        Discount $payoff$ to present value using $e^{-r \cdot T}$
    end for
    $C = \text{mean}(payoff)$
    return C
end procedure

Algorithm 7 Analytical Approximation Using Broadie, Glasserman, and Kou's Technique

procedure BGKApproximation(S_0, K, b, r, q, T, m, σ)
    Adjust barrier $H = b \cdot \exp(-0.5826 \cdot \sigma \cdot \sqrt{T/m})$
    Compute $l = \frac{r-q+\sigma^2/2}{\sigma^2}$
    Calculate $y = \frac{\log((H^2)/(S_0 \cdot K))}{\sigma \cdot \sqrt{T}} + l \cdot \sigma \cdot \sqrt{T}$
    Calculate $y2 = y - \sigma \cdot \sqrt{T}$
    Compute probabilities using normal CDF: $N_y = \text{norm.cdf}(y)$, $N_y2 = \text{norm.cdf}(y2)$
    Calculate down-and-in call price:
    $C_{di} = S_0 \cdot e^{-q \cdot T} \cdot (H/S_0)^{2 \cdot l} \cdot N_y - K \cdot e^{-r \cdot T} \cdot (H/S_0)^{2 \cdot l - 2} \cdot N_y 2$
    return C_{di}
end procedure

4.2 Results: Task 3

Table 4: Barrier Option for 12 periods

Method	$\sigma = 20\%$	$\sigma = 40\%$
Standard MC	0.036	1.343
BGK	0.034	1.344

Table 5: Barrier Option for 52 periods

Method	$\sigma = 20\%$	$\sigma = 40\%$
Standard MC	0.055	1.987
BGK	0.059	1.973

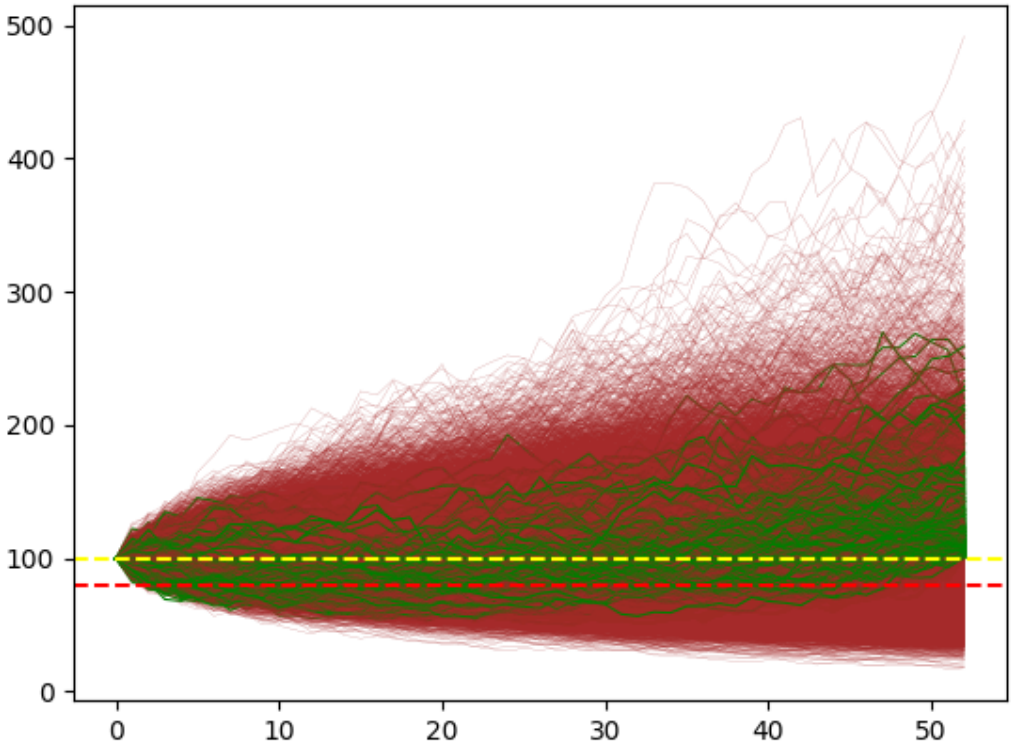


Figure 4: Plot for the 20 000 simulated stock paths with volatility = 40 % and 52 periods. The green lines represent the paths that are going to be executed at time T.
This plot is included to showcase the different stock paths and highlight the paths that result in a nonzero payoff.

4.3 Discussion: Task 3

Why is the price higher when the volatility is higher?
The price is higher when the volatility is higher because a higher volatility increases the amplitude of the generated paths. This leads to a higher probability of breaching the barrier and it also amplifies the payoff of the nonzero payoff paths.

5 Task 4: Basket options

5.1 Method/pseudocode for Task 4

Algorithm 8 Monte Carlo Simulation for Basket Option Pricing

procedure BasketOptionPricing(S_0, r, σ, T, K, ρ, n)
    Define $dt = \frac{T}{n}$ ▷ Time step
    Initialize arrays $S_1[0] = S_0$ and $S_2[0] = S_0$ for the two assets
    Generate correlated random variables Z_1 and Z_2 using ρ
    for $i = 1$ to n **do**
        Simulate asset paths using GBM:
        $S_1[i] = S_1[i-1] \cdot e^{(r-0.5\cdot\sigma^2)\cdot dt+\sigma\cdot\sqrt{dt}\cdot Z_1[i]}$
        $S_2[i] = S_2[i-1] \cdot e^{(r-0.5\cdot\sigma^2)\cdot dt+\sigma\cdot\sqrt{dt}\cdot Z_2[i]}$
    end for
    Compute the average price of the two assets at maturity:
    $S_{\text{avg}} = \frac{1}{2}(S_1[n] + S_2[n])$
    Calculate the call and put payoffs:
    Call payoff $= \max(S_{\text{avg}} - K, 0)$
    Put payoff $= \max(K - S_{\text{avg}}, 0)$
    Discount the payoffs to present value:
    $PV_{\text{call}} = e^{-r\cdot T} \cdot$ Call payoff
    $PV_{\text{put}} = e^{-r\cdot T} \cdot$ Put payoff
    Compute the average discounted payoff over all simulations:
    $C = \text{mean}(PV_{\text{call}})$
    $P = \text{mean}(PV_{\text{put}})$
    return C, P
end procedure

5.2 Results: Task 4

The following tables compares the standard MC for both a call and put basket option at various strike prices with specific ρ values:

Table 6: Basket option Call

Method/K	K = 70	K = 100	K = 130
$\rho = 0$	32.020	7.164	0.338
$\rho = 0.5$	32.064	8.266	0.743

Table 7: Basket option put

Method/K	K = 70	K = 100	K = 130
$\rho = 0$	0.011	4.233	26.523
$\rho = 0.5$	0.0512	5.255	27.000

5.3 Discussion: Task 4

Why are the prices lower than in problem 1?
Both the put and call prices for the basket options are based on the correlation between the two underlying stocks and thereby their correlation. In task 1 we had one stock as underlying asset. If the correlation between the stocks in the barrier option was equal to one we would have similar prices as in task 1. But since the correlation is smaller than 1 we get smaller option prices.

6 Task 5: Autocall on Four Underlying Stocks

6.1 Method/ Pseudocode for Task 5

Algorithm 9 Simulate Autocall Prices

procedure SIMULATEAUTOCALL(*paths*, *periods*, r, *sigma*, *nominal*, *autocall_barrier*, *coupon_barrier*, *risk_barrier*, *rho*)
    Initialize stock prices S_1, S_2, S_3, S_4 to S_0
    Compute the Cholesky decomposition L of the correlation matrix *rho*
    Initialize *prices* array to zero for each path
    $dt \leftarrow 1$ ▷ Each period is one year in this context
    for $i \leftarrow 1$ to *paths* **do**
        Reset S_1, S_2, S_3, S_4 to S_0
        Initialize *accumulated_coupons* $\leftarrow 0$
        for $j \leftarrow 1$ to *periods* **do**
            Generate correlated random variables Z
            Update each stock S_k using GBM formula
            $WPS \leftarrow \min(S_1, S_2, S_3, S_4)$ ▷ Worst Performing Stock
            Calculate the payoffs based on the different barriers and the WPS.
            Discount the final payoff.
        end for
    end for
    mean_price $\leftarrow$ mean(*prices*)
    return *mean_price*
end procedure

Algorithm 10 Determine Coupon Rates for Target Price

procedure FINDCOUPONRATE(*paths*, *periods*, r, *sigma*, *nominal*, *autocall_barrier*, *coupon_barrier*, *risk_barrier*, *rho*, *target_price*, *coupon_rates*)
    Initialize *min_rate* $\leftarrow \min($*coupon_rates*$)$
    Initialize *max_rate* $\leftarrow \max($*coupon_rates*$)$
    Initialize *tolerance* $\leftarrow 0.01$
    while *max_rate* $-$ *min_rate* $>$ *tolerance* **do**
        mid_rate $\leftarrow$ (*min_rate* $+$ *max_rate*)$/2$
        prices $\leftarrow$ simulate_autocall(*paths*, *periods*, r, *sigma*, *nominal*, *autocall_barrier*, *coupon_barri*
        mean_price $\leftarrow$ mean(*prices*)
        if *mean_price* $<$ *target_price* **then**
            min_rate $\leftarrow$ *mid_rate*
        else
            max_rate $\leftarrow$ *mid_rate*
        end if
    end while
    final_rate $\leftarrow$ (*min_rate* $+$ *max_rate*)$/2$
    return *final_rate*
end procedure

6.2 Results: Task 5

Table 8: Autocall with Four Underlying Stocks

	$\rho = 0$	$\rho = 0.5$
Standard MC	92.531	98.632
Coupon rate	0.197	0.113

6.3 Discussion: Task 5

Why does higher correlation imply a higher price?

Higher correlation in underlying assets in an autocall usually leads to a higher price. This is because when assets are highly correlated they move up or down together which increases the likelihood that they will all surpass the autocall barrier at the same time. This movement makes it more likely that the autocall will be triggered early giving investors their investment back with a premium in a shorter period.

7 Python Code to solve all tasks

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import math
from scipy.stats import norm, lognorm
from scipy.stats.mstats import gmean
from ctypes import sizeof
from importlib.resources import path
from pickle import HIGHEST_PROTOCOL
from turtle import pu

"""

s0 = 100
r = 0.03
T = 1
t = 0
sigma = 0.2
K1 = 70
K2 = 100
K3 = 130

#1 a)

def BS(strike):
    d1 = 1/(sigma * (math.sqrt(T-t))) * (math.log(s0/strike) + (
        r + 0.5 * (sigma * sigma))*(T-t))
    d2 = d1 - sigma * math.sqrt(T-t)
    call = s0 *  norm.cdf(d1) - math.exp((-r)* (T-t)) * strike *
        norm.cdf(d2)
    put = call - s0 + strike * math.exp((-r)* (T-t))
    return call, put



print("call price, put price for K = 70: ", BS(K1))
print("call price, put price for K = 100: ", BS(K2))
print("call price, put price for K = 130: ", BS(K3))


"""
# 1 b)
```

```
"""
S_0 = 100
r = 0.03
intrest_rate = np.linspace(0,0.1, num=10)
sigma1 = 0.2
sigma = np.linspace(0.01, 0.5, num=49)
K = [70, 100, 130]
T = 1
time_to_maturity = np.linspace(1/12, 5, num=59)
t = 0
k = 0


def BS(type):
    S_0 = 100
    r = 0.03
    intrest_rate = np.linspace(0,0.1, num=10)
    sigma1 = 0.2
    sigma = np.linspace(0.01, 0.5, num=49)
    K = [70, 100, 130]
    T = 1
    time_to_maturity = np.linspace(1/12, 5, num=59)
    t = 0
    k = 1


    call_prices = []
    put_prices = []
    if type == "sigma":
        for i in range(len(sigma)):
            d_1 = d_1_calc(sigma[i], T, t, K, k)
            d_2 = d_2_calc(d_1, sigma[i], T, t)
            N_d1 = n_calc(d_1)
            N_d2 = n_calc(d_2)
            Black_Scholes_price = S_0 * N_d1 - math.exp(-r*(T-t))
                * K[k] * N_d2
            PC_Parity_Put_Price = Black_Scholes_price - S_0 + K[k
               ] * math.exp(-r*(T-t))
            call_prices.append(Black_Scholes_price)
            put_prices.append(PC_Parity_Put_Price)
    elif type == "time_to_maturity":
        for i in range(len(time_to_maturity)):
            d_1 = d_1_calc(sigma1, time_to_maturity[i], t, K, k)
            d_2 = d_2_calc(d_1, sigma1, time_to_maturity[i], t)
            N_d1 = n_calc(d_1)
            N_d2 = n_calc(d_2)
            Black_Scholes_price = S_0 * N_d1 - math.exp(-r*(
```

```
                time_to_maturity[i]-t)) * K[k] * N_d2
            PC_Parity_Put_Price = Black_Scholes_price - S_0 + K[k
                ] * math.exp(-r*(time_to_maturity[i]-t))
            call_prices.append(Black_Scholes_price)
            put_prices.append(PC_Parity_Put_Price)
    elif type == "intrest_rate":
        for i in range(len(intrest_rate)):
            d_1 = d_1_calc(sigma1, T, t, K, k)
            d_2 = d_2_calc(d_1, sigma1, T, t)
            N_d1 = n_calc(d_1)
            N_d2 = n_calc(d_2)
            Black_Scholes_price = S_0 * N_d1 - math.exp(-(
                intrest_rate[i])*(T-t)) * K[k] * N_d2
            PC_Parity_Put_Price = Black_Scholes_price - S_0 + K[k
                ] * math.exp(-(intrest_rate[i])*(T-t))
            call_prices.append(Black_Scholes_price)
            put_prices.append(PC_Parity_Put_Price)




    return call_prices, put_prices

def d_1_calc(sigma, T, t, K, k):
    d_1 = 1 / (sigma * math.sqrt(T-t)) * (math.log(S_0 / K[k]) +
        (r +1/2 * sigma * sigma )*(T-t))
    return d_1


def d_2_calc(d_1, sigma, T, t):
    d_2 = d_1 - sigma * math.sqrt(T-t)
    return d_2


def n_calc(x):
    if x > 0:
        N = norm.cdf(x)
    else:
        N = 1 - norm.cdf(-x)
    return N


# Plots for Task 1b varying volatility
def Task_1_b(type):
    #sub_sub_task = ["sigma", "time_to_maturity", "intrest_rate"]
    call_prices, put_prices = BS(type)
    if type == "sigma": # sub_sub_task[0]:
        plt.plot(sigma,  call_prices, label='Call Price')
```

```
        plt.plot(sigma, put_prices, label='Put Price')
        plt.xlabel('volatility')
        plt.ylabel('Price')
        plt.title('Black-Scholes Call and Put Prices for
            different volatilities')
    elif type == "time_to_maturity": # sub_sub_task[1]:
        plt.plot(time_to_maturity, call_prices, label='Call
            Price')
        plt.plot(time_to_maturity, put_prices, label='Put Price')
        plt.xlabel('time to maturity')
        plt.ylabel('Price')
        plt.title('Black-Scholes Call and Put Prices for
            different time to maturities')
    elif type == "intrest_rate": # sub_sub_task[2]:
        plt.plot(intrest_rate, call_prices, label='Call Price')
        plt.plot(intrest_rate, put_prices, label='Put Price')
        plt.xlabel('intrest rate')
        plt.ylabel('Price')
        plt.title('Black-Scholes Call and Put Prices for
            different intrest rates')
    plt.legend()
    plt.show()


# Task 1b
#Task_1_b("sigma")
#Task_1_b("time_to_maturity")
Task_1_b("intrest_rate")


"""
"""
# 1 c)

def q1c(K):
    s0 = 100
    r = 0.03
    sigma = 0.2
    T = 1
    n = 20000

    z = np.random.standard_normal((n, 1))


    stockprices1 = []
    for i in z:
        ST = s0 * math.exp(((r - (0.5 * (sigma * sigma))) * T) +
            (sigma * math.sqrt(T) * i))
```

```
    stockprices1.append(ST)


payoffscall = []

for j in stockprices1:
    if j - K > 0:

        payoff = j - K
        payoffscall.append(payoff)
    else:
        zero  = 0
        payoffscall.append(zero)

payoffsput = []

for j2 in stockprices1:
    if K - j2 > 0:

        payoff2 = K - j2
        payoffsput.append(payoff2)
    else:
        zero  = 0
        payoffsput.append(zero)

dispayoffcall = []
dispayoffput = []

for g in payoffscall:
    c = math.exp(-r * T) * g
    dispayoffcall.append(c)

dispayoff_ncall = np.mean(dispayoffcall)

for g2 in payoffsput:
    c2 = math.exp(-r * T) * g2
    dispayoffput.append(c2)

dispayoff_nput = np.mean(dispayoffput)

sigmanew = np.std(dispayoffcall)

u = 1.96 * (sigmanew / math.sqrt(n))

upper  = dispayoff_ncall + u
lower = dispayoff_ncall - u
intervalcall = [lower, dispayoff_ncall, upper]
```

```
    sigmanew2 = np.std(dispayoffput)

    u2 = 1.96 * (sigmanew2 / math.sqrt(n))

    upper2  = dispayoff_nput + u2
    lower2 = dispayoff_nput - u2
    intervalput = [lower2, dispayoff_nput, upper2]


    return dispayoff_ncall, intervalcall, dispayoff_nput,
       intervalput



print("Strike 70 ", q1c(70))
print("Strike 100 ", q1c(100))
print("Strike 130 ", q1c(130))


"""

"""

# 1 d)

def q1d(K):

    s0 = 100
    r = 0.03
    sigma = 0.2
    T = 1
    t = 0
    n = 10000


    z = np.random.standard_normal((n, 1))

    stockprices = []
    for i in z:
        yplus = s0 * math.exp(((r - (0.5 * (sigma * sigma))) * T
           ) + (sigma * math.sqrt(T) * i))
        yminus = s0 * math.exp(((r - (0.5 * (sigma * sigma))) *
           T) + (sigma * math.sqrt(T) * (-i)))
        stockprices.append(yplus)
        stockprices.append(yminus)


    payoffscall = []
```

```
for j in stockprices:
    if j - K > 0:

        payoff = j - K
        payoffscall.append(payoff)
    else:
        zero  = 0
        payoffscall.append(zero)


dispayoffcall = []

for g in payoffscall:
    c = math.exp(-r * T) * g
    dispayoffcall.append(c)

dispayoff_ncall = np.sum(dispayoffcall)/(2*n)

payoffsput = []

for j2 in stockprices:
    if K - j2 > 0:

        payoff = K - j2
        payoffsput.append(payoff)
    else:
        zero  = 0
        payoffsput.append(zero)

dispayoffput = []

for g2 in payoffsput:
    c2 = math.exp(-r * T) * g2
    dispayoffput.append(c2)

dispayoff_nput = np.sum(dispayoffput)/(2*n)


sigmacall = np.std(dispayoffcall)

u = 1.96 * (sigmacall / math.sqrt(2*n))

lower = dispayoff_ncall - u
upper = dispayoff_ncall + u

intervalcall = [lower, dispayoff_ncall, upper]
```

```
    sigmaput = np.std(dispayoffput)

    u2 = 1.96 * (sigmaput / math.sqrt(2*n))

    lower2 = dispayoff_nput - u2
    upper2 = dispayoff_nput + u2

    intervalput = [lower2, dispayoff_nput, upper2]

    return dispayoff_ncall, intervalcall, dispayoff_nput,
       intervalput

print("Strike 70: ", q1d(70))
print("Strike 100: ", q1d(100))
print("Strike 130: ", q1d(130))

"""

"""

# 1 e)

s0 = 100
r = 0.03
sigma = 0.2
T = 1
t = 0
K = 100

pricesAV = []
pricesSMC = []


######AV
start1 = 5000
stop2 = 50000
step3 = 5000

for n in range(start1, stop2 + 1, step3):


    z = np.random.standard_normal((n, 1))

    stockpricesAV = []
    for i in z:
        STplus = s0 * math.exp(((r - (0.5 * (sigma * sigma))) *
           T) + (sigma * math.sqrt(T) * i))
```

```
        STminus = s0 * math.exp(((r - (0.5 * (sigma * sigma))) *
            T) + (sigma * math.sqrt(T) * (-i)))
        stockpricesAV.append(STplus)
        stockpricesAV.append(STminus)



    payoffsAV = []

    for j in stockpricesAV:
        if j - K > 0:
            payoff1 = j - K
            payoffsAV.append(payoff1)
        else:
            zero  = 0
            payoffsAV.append(zero)



    dispayoffAV = []

    for g in payoffsAV:
        c = math.exp(-r * T) * g
        dispayoffAV.append(c)

    dispayoff_n1 = np.sum(dispayoffAV)/(2*n)

    pricesAV.append(dispayoff_n1)

 ######SMC
start4 = 10000
stop5 = 100000
step6 = 10000

for n2 in range(start4, stop5 + 1, step6):

    z = np.random.standard_normal((n2, 1))


    stockpricesSMC = []
    for ii in z:
        ST = s0 * math.exp(((r - (0.5 * (sigma * sigma))) * T) +
            (sigma * math.sqrt(T) * ii))
        stockpricesSMC.append(ST)

    payoffsSMC = []

    for jj in stockpricesSMC:
```

```
        if jj - K > 0:
            payoff2 = jj - K
            payoffsSMC.append(payoff2)
        else:
            zero  = 0
            payoffsSMC.append(zero)



    dispayoffSMC = []

    for gg in payoffsSMC:
        c2 = math.exp(-r * T) * gg
        dispayoffSMC.append(c2)

    dispayoff_n2 = np.mean(dispayoffSMC)

    pricesSMC.append(dispayoff_n2)


print("AV prices: ", pricesAV, "SMC prices: ", pricesSMC)


def BS(strike):
    d1 = 1/(sigma * (math.sqrt(T-t))) * (math.log(s0/strike) + (
        r + 0.5 * (sigma * sigma))*(T-t))
    d2 = d1 - sigma * math.sqrt(T-t)
    BS = s0 *  norm.cdf(d1) - math.exp((-r)* (T-t)) * strike *
        norm.cdf(d2)
    return BS

BS1e = BS(K)



BSvector = np.ones(10) * BS1e

#Plot 1e

plt.figure(figsize=(10, 5))
plt.plot(range(start4, stop5+1, step6), pricesAV, label='priceAV
    ')
plt.plot(range(start4, stop5+1, step6), pricesSMC, label='
    priceSMC')
plt.plot(range(start4, stop5+1, step6), BSvector, label='Black
    scholes')
plt.title('Comparison plot between SMC-price, AV-price and Black
    Scholes')
```

```
plt.xlabel('Number of simulations')
plt.ylabel('Option Price')
plt.legend()
plt.grid(True)
plt.show()

"""

"""
#2 i)


r = 0.03
sigma = 0.2
s0 = 100


def asiancall(K, m):

    ti = []

    for i in range(1,m+1,1):
        t = (1/m)* i
        ti.append(t)

        T = np.sum(ti)/m
        Maturity = ti[-1]

        V = []

    for ii in range(1,m+1,1):
        sigmaVariable = (2*ii-1) * ti[-1 + 1 - ii]
        V.append(sigmaVariable)

        Vinsigmaformula = np.sum(V)


    sigmahattsquare = (np.square(sigma)/(np.square(m)*T)) *
       Vinsigmaformula


    d = (0.5 * np.square(sigma)) - (0.5 * sigmahattsquare)

    d1 = (1/(np.sqrt(sigmahattsquare) * (math.sqrt(T)))) * (math
       .log(s0/K) + (r - d + (0.5 * sigmahattsquare))*T)
    d2 = d1 - (np.sqrt(sigmahattsquare) * math.sqrt(T))
    ST = math.exp((-r)* (Maturity-T)) * (math.exp((-d) * T) * s0
        *  norm.cdf(d1) -  math.exp((-r) * T) * K * norm.cdf(d2)
```

```
        )

    return ST


print("Exact price of asian call strike = 70 and m = 12: ",
   asiancall(70,12))
print("Exact price of asian call strike = 100 and m = 12: ",
   asiancall(100,12))
print("Exact price of asian call strike = 130 and m = 12: ",
   asiancall(130,12))
print("Exact price of asian call strike = 70 and m = 52: ",
   asiancall(70,52))
print("Exact price of asian call strike = 100 and m = 52: ",
   asiancall(100,52))
print("Exact price of asian call strike = 130 and m = 52: ",
   asiancall(130,52))

"""


"""
# 2 ii a & b)

def q2(K,m):

    r = 0.03
    sigma = 0.2
    s0 = 100
    T2 = 1

    ti = []

    for i in range(1,m+1,1):
        t = (1/m)* i
        ti.append(t)

        T = np.sum(ti)/m
        Maturity = ti[-1]

        V = []

    for ii in range(1,m+1,1):
        sigmaVariable = (2*ii-1) * ti[-1 + 1 - ii]
        V.append(sigmaVariable)
        Vinsigmaformula = np.sum(V)
```

```
sigmahattsquare = (np.square(sigma)/(np.square(m)*T)) *
    Vinsigmaformula


d = (0.5 * np.square(sigma)) - (0.5 * sigmahattsquare)

d1 = (1/(np.sqrt(sigmahattsquare) * (math.sqrt(T)))) * (math
    .log(s0/K) + (r - d + (0.5 * sigmahattsquare))*T)
d2 = d1 - (np.sqrt(sigmahattsquare) * math.sqrt(T))
ST1 = math.exp((-r)* (Maturity-T)) * (math.exp((-d) * T) *
    s0 *  norm.cdf(d1) -  math.exp((-r) * T) * K * norm.cdf(
    d2))

n = 20000

dt = T2 / m

z = np.random.standard_normal((n, m + 1))

stock_prices = s0 * np.ones((n, m + 1))
stock_prices[:,0] = s0

for o in range((n)):
    for t in range(1, m + 1):
        stock_prices[o, t] = stock_prices[o, t - 1] * np.exp
            (((r - 0.5 * sigma ** 2) * dt + (sigma * np.sqrt(
            dt)) * z[o, t]))


C1 = gmean(stock_prices[:, 1:], axis = 1)
payoffsCV = np.maximum(C1-K, 0) * math.exp(-r * T2)


C2 = np.mean(stock_prices[:, 1:], axis = 1)
payoffsSMC = np.maximum(C2-K,0) * math.exp(-r * T2)


Geosim = np.mean(payoffsCV)
SMC = np.mean(payoffsSMC)

#b = (np.cov(dispayoffCV,dispayoffSMC)[0,1]) / np.var(
    dispayoffCV)
b = sum((payoffsCV - Geosim) * (payoffsSMC - SMC)) / sum((
    payoffsSMC - Geosim) ** 2)

Controlvariate = (SMC - b*(Geosim - ST1))
```

```
    return Geosim, SMC, Controlvariate

print("Geosim, SMC and controlvariate for strike 70 and m = 12
    ", q2(70, 12))
print("Geosim, SMC and controlvariate for strike 100 and m = 12
    ", q2(100, 12))
print("Geosim, SMC and controlvariate for strike 130 and m = 12
    ", q2(130, 12))
print("Geosim, SMC and controlvariate for strike 70 and m = 52
    ", q2(70, 52))
print("Geosim, SMC and controlvariate for strike 70 and m = 52
    ", q2(100, 52))
print("Geosim, SMC and controlvariate for strike 130 and m = 52
    ", q2(130, 52))

"""

"""
# 2 ii) c)

resSMC = []
resCV = []
for n in range(10000, 100001, 10000):
    r = 0.03
    sigma = 0.2
    s0 = 100
    T2 = 1
    m = 52
    K = 100
    ST1 = 5.1668732803866115 #from 2ii a)

    dt = T2 / m

    z = np.random.standard_normal((n, m + 1))

    stock_prices = s0 * np.ones((n, m + 1))
    stock_prices[:,0] = s0

    for o in range((n)):
        for t in range(1, m + 1):
            stock_prices[o, t] = stock_prices[o, t - 1] * np.exp
                (((r - 0.5 * sigma ** 2) * dt + (sigma * np.sqrt(
                dt)) * z[o, t]))


    C1 = gmean(stock_prices[:, 1:], axis = 1)
    payoffsCV = np.maximum(C1-K, 0) * math.exp(-r * T2)
```

```
        C2 = np.mean(stock_prices[:, 1:], axis = 1)
        payoffsSMC = np.maximum(C2-K,0) * math.exp(-r * T2)


        Geosim = np.mean(payoffsCV)
        SMC = np.mean(payoffsSMC)

        #b = (np.cov(dispayoffCV,dispayoffSMC)[0,1]) / np.var(
            dispayoffCV)
        b = sum((payoffsCV - Geosim) * (payoffsSMC - SMC)) / sum((
            payoffsSMC - Geosim) ** 2)

        Controlvariate = (SMC - b*(Geosim - ST1))
        resSMC.append(SMC)
        resCV.append(Controlvariate)

plt.figure(figsize=(10, 5))
plt.plot(range(10000, 100000 + 1, 10000), resSMC, label='SMC')
plt.plot(range(10000, 100000 + 1, 10000), resCV, label='CV-price
    ')
plt.title('Comparison plot for K = 100 M = 52')
plt.xlabel('Number of simulations')
plt.ylabel('Option Price')
plt.legend()
plt.grid(True)
plt.show()

"""

"""

# Task 3a


paths = 20000; T = 1; periods = [12, 52]; r = 0.03; sigma =
    [0.2, 0.4]; barrier = 80; K = 100
for m in periods:
    for s in sigma:
        dt = 1/m
        Z = np.random.standard_normal((paths, T*m + 1))
        S = 100*np.ones((paths, T*m + 1))
        for n in range(paths):
            for t in range(1, T*m + 1):
                S[n, t] = S[n, t-1] * np.exp((r-0.5*s**2)*dt + np
                    .sqrt(dt)*s*Z[n,t])
```

```
        payoff = np.zeros(paths)
        for n in range(paths):
            if np.any(S[n] <= barrier):
                payoff[n] = np.exp(-r*T) * np.maximum(S[n, -1] -
                    100, 0)  # Set payoff to 0 if never below
                    barrier


        C = np.mean(payoff)
        print("Barrier Down and in Call option price estimate for
            ",paths, " simulated paths with sigma = ",s, " and
           periods = ",m, " is = {:.3f}".format(C))



# Task 3b


sigma = [0.2, 0.4]; periods = [12, 52]; T = 1; K = 100; S_0 =
    100; q = 0; r = 0.03


for m in periods:
    for s in sigma:
        H = 80 * np.exp(-0.5826 * s * np.sqrt((T/m)))
        l = (r - q + (s**2) / 2) / (s**2)
        y = (math.log((H**2) / (S_0 * K)) / (s * np.sqrt(T))) + l
            * s * np.sqrt(T)
        y2 = y - s * np.sqrt(T)
        N_y = norm.cdf(y)
        N_y2 = norm.cdf(y2)


        C_di = S_0 * np.exp(-q*T) * (H/S_0)**(2*l) * N_y - K * np
            .exp(-r*T) * (H/S_0)**(2*l-2) * N_y2
        print("Barrier Down and in Call option price for sigma =
            ", s, " and periods = ", m, " is = {:.3f}".format(C_di
            ))


# Plots for 3a

paths = 20000; T = 1; m = 12; r = 0.03; s = 0.2; barrier = 80; K
    = 100


dt = 1/m
```

```
Z = np.random.standard_normal((paths, T*m + 1))
S = 100*np.ones((paths, T*m + 1))
for n in range(paths):
    for t in range(1, T*m + 1):
        S[n, t] = S[n, t-1] * np.exp((r-0.5*s**2)*dt + np.sqrt(dt
            )*s*Z[n,t])


payoff = np.zeros(paths)
for n in range(paths):
    if np.any(S[n] <= barrier):
        payoff[n] = np.exp(-r*T) * np.maximum(S[n, -1] - 100, 0)
            # Set payoff to 0 if never below barrier


C = np.mean(payoff)
print("Barrier Down and in Call option price estimate for ",
    paths, " simulated paths with sigma = ",s, " and periods = ",
    m, " is = {:.3f}".format(C))




colors = ['green' if (np.any(S[n] < barrier) and S[n, -1] > 100)
      else 'brown' for n in range(paths)]
for n in range(paths):
    #if colors[n]=='green':
    if colors[n] == 'green':
        plt.plot(S[n], color=colors[n], linewidth = 0.8)
    elif colors[n] == 'brown':
        plt.plot(S[n], color=colors[n], linewidth = 0.1)
plt.axhline(y=barrier, color='r', linestyle='--') # Add red
    line at barrier level
plt.axhline(y=K, color='yellow', linestyle='--') # Add yellow
    line at strike K level
plt.title("The " + str(np.sum(payoff > 0)) + " green paths out
    of the total 20 000 paths are going to be executed at time T
    ")
plt.show()


"""



"""
```

```
# 4 a)


def q4a(K,p):
    r = 0.03
    sigma = 0.2
    s0 = 100
    T = 1
    n = 100000
    n_steps = 1


    dt = T / n_steps

    z1 = np.random.standard_normal((n, n_steps + 1))

    Covariance_matrix = np.array([[sigma**2,sigma*sigma*p], [
       sigma*sigma*p, sigma**2]])

    V = np.linalg.cholesky(Covariance_matrix)

    z2 = np.matmul(z1, V)


    stock_pricesx = s0 * np.ones((n, n_steps + 1))
    stock_pricesx[:,0] = s0
    stock_pricesy = s0 * np.ones((n, n_steps + 1))
    stock_pricesy[:,0] = s0

    for i in range((n)):
        stock_pricesx[i] = stock_pricesx[i] * np.exp(((r - 0.5 *
            sigma ** 2) * dt + ( np.sqrt(dt)) * z2[i,0]))
        stock_pricesy[i] = stock_pricesy[i] * np.exp(((r - 0.5 *
            sigma ** 2) * dt + ( np.sqrt(dt)) * z2[i,1]))



    c = 0.5 * (stock_pricesx + stock_pricesy)

    callpayoffs = np.maximum(c-K,0) * math.exp(-r * T)

    callres = np.mean(callpayoffs)


    return callres
```

```
print("CALL K = 70, P = 0.5", q4a(70,0.5))
print("CALL K = 100, P = 0.5", q4a(100,0.5))
print("CALL K = 130, P = 0.5", q4a(130,0.5))
print("CALL K = 70, P = 0", q4a(70,0.))
print("CALL K = 100, P = 0", q4a(100,0))
print("CALL K = 130, P = 0", q4a(130,0))


# 4 b)

def q4b(K,p):
    r = 0.03
    sigma = 0.2
    s0 = 100
    T = 1
    n = 100000
    n_steps = 1


    dt = T / n_steps

    z1 = np.random.standard_normal((n, n_steps + 1))

    Covariance_matrix = np.array([[sigma**2,sigma*sigma*p], [
       sigma*sigma*p, sigma**2]])

    V = np.linalg.cholesky(Covariance_matrix)

    z2 = np.matmul(z1, V)


    stock_pricesx = s0 * np.ones((n, n_steps + 1))
    stock_pricesx[:,0] = s0
    stock_pricesy = s0 * np.ones((n, n_steps + 1))
    stock_pricesy[:,0] = s0

    for i in range((n)):
        stock_pricesx[i] = stock_pricesx[i] * np.exp(((r - 0.5 *
            sigma ** 2) * dt + ( np.sqrt(dt)) * z2[i,0]))
        stock_pricesy[i] = stock_pricesy[i] * np.exp(((r - 0.5 *
            sigma ** 2) * dt + ( np.sqrt(dt)) * z2[i,1]))



    c = 0.5 * (stock_pricesx + stock_pricesy)

    putpayoff = np.maximum(K-c,0) * math.exp(-r * T)
```

```
    putres = np.mean(putpayoff)


    return putres

print("PUT K = 70, P = 0.5", q4b(70,0.5))
print("PUT K = 100, P = 0.5", q4b(100,0.5))
print("PUT K = 130, P = 0.5", q4b(130,0.5))
print("PUT K = 70, P = 0", q4b(70,0.))
print("PUT K = 100, P = 0", q4b(100,0))
print("PUT K = 130, P = 0", q4b(130,0))

"""

"""
# 5 a)


# Parameters
S_0 = 100
r = 0.03
sigma = 0.2
nominal = 100
autocall_barrier = 100
coupon_barrier = 95
risk_barrier = 70
paths = 20000
periods = 7
rho_1 = np.eye(4)
rho_2 = np.ones((4, 4)) * 0.5 + np.eye(4) * (1 - 0.5)
coupon_rates1 = np.linspace(0.15,0.25, num=50)
coupon_rates2 = np.linspace(0.05,0.15, num=50)


# Function to simulate autocall prices using Monte Carlo
    simulation
def simulate_autocall(paths, periods, r, sigma, nominal,
    autocall_barrier, coupon_barrier, risk_barrier, rho):
    # Generate correlated random numbers
    Z = np.random.standard_normal((4, paths, periods))
    L = np.linalg.cholesky(rho)
    Z_corr = np.einsum('ij,jkl->ikl', L, Z)
    # Initialize prices array
    prices = np.zeros(paths)
    # Simulate autocall prices for each path
    WPS_list = np.zeros((paths, periods+1))
    WPS_list[:,0] = 100
    dt = 1
```

```
    for i in range(paths):
        S1 = S2 = S3 = S4 = S_0
        accumulated_coupons = 0
        for j in range(periods):
            #dt=1/(j+1)
            S1 *= np.exp((r - 0.5 * sigma ** 2) * dt + sigma * np
               .sqrt(dt) * Z_corr[0, i, j])
            S2 *= np.exp((r - 0.5 * sigma ** 2) * dt + sigma * np
               .sqrt(dt) * Z_corr[1, i, j])
            S3 *= np.exp((r - 0.5 * sigma ** 2) * dt + sigma * np
               .sqrt(dt) * Z_corr[2, i, j])
            S4 *= np.exp((r - 0.5 * sigma ** 2) * dt + sigma * np
               .sqrt(dt) * Z_corr[3, i, j])
            WPS = min(S1, S2, S3, S4)
            WPS_list[i,j+1] = WPS


            if WPS > autocall_barrier:
                prices[i] += np.exp(-r * (j+1)) * nominal *(1.1 +
                    accumulated_coupons * 0.1)
                break
            elif WPS < autocall_barrier and WPS > coupon_barrier:
                if j != 6:
                    prices[i] += np.exp(-r * (j+1)) * nominal
                       *(0.1 + accumulated_coupons * 0.1)
                    accumulated_coupons = 0
                else:
                    prices[i] += np.exp(-r * (j+1)) * nominal *(1
                        + 0.1 + accumulated_coupons * 0.1)
                    break
            else:
                if j != 6:
                    if WPS > risk_barrier:
                        accumulated_coupons += 1
                    elif WPS < risk_barrier:
                        prices[i] += np.exp(-r * (j+1)) * nominal
                            * 0.9# + accumulated_coupons * 0.1)
                        break
                elif j == 6:
                    if WPS > risk_barrier:
                        prices[i] += np.exp(-r * (j+1)) * nominal
                    else:
                        prices[i] += np.exp(-r * (j+1)) * 0.9 *
                           nominal

    return prices, WPS_list

"""
```

```
"""

#5b)

# Simulate autocall prices for both sets of correlations
prices_1, WPS_list1 = simulate_autocall(paths, periods, r, sigma
    , nominal, autocall_barrier, coupon_barrier, risk_barrier,
    rho_1)
prices_2, WPS_list2 = simulate_autocall(paths, periods, r, sigma
    , nominal, autocall_barrier, coupon_barrier, risk_barrier,
    rho_2)


# Calculate mean prices
mean_price_1 = np.mean(prices_1)
mean_price_2 = np.mean(prices_2)
print("Price 1: ", mean_price_1)
print("Price 2: ", mean_price_2)

def simulate_coupon_rate(paths, periods, r, sigma, nominal,
    autocall_barrier, coupon_barrier, risk_barrier, rho,
    coupon_rate):

    # Generate correlated random numbers
    Z = np.random.standard_normal((4, paths, periods))
    L = np.linalg.cholesky(rho)
    Z_corr = np.einsum('ij,jkl->ikl', L, Z)
    # Initialize prices array
    prices = np.zeros(paths)
    # Simulate autocall prices for each path
    WPS_list = np.zeros((paths, periods+1))
    dt = 1
    for i in range(paths):
        S1 = S2 = S3 = S4 = S_0
        accumulated_coupons = 0
        for j in range(periods):
            #dt=1/(j+1)
            S1 *= np.exp((r - 0.5 * sigma ** 2) * dt + sigma * np
                .sqrt(dt) * Z_corr[0, i, j])
            S2 *= np.exp((r - 0.5 * sigma ** 2) * dt + sigma * np
                .sqrt(dt) * Z_corr[1, i, j])
            S3 *= np.exp((r - 0.5 * sigma ** 2) * dt + sigma * np
                .sqrt(dt) * Z_corr[2, i, j])
            S4 *= np.exp((r - 0.5 * sigma ** 2) * dt + sigma * np
                .sqrt(dt) * Z_corr[3, i, j])
            WPS = min(S1, S2, S3, S4)

            if WPS > autocall_barrier:
```

```
                prices[i] += np.exp(-r * (j+1)) * nominal *(1+
                    coupon_rate + accumulated_coupons *
                    coupon_rate)
                break
            elif WPS < autocall_barrier and WPS > coupon_barrier:
                if j != 6:
                    prices[i] += np.exp(-r * (j+1)) * nominal *(
                        coupon_rate + accumulated_coupons *
                        coupon_rate)
                    accumulated_coupons = 0
                else:
                    prices[i] += np.exp(-r * (j+1)) * nominal *(1
                        + coupon_rate + accumulated_coupons *
                        coupon_rate)
                    break
            else:
                if j != 6:
                    if WPS > risk_barrier:
                        accumulated_coupons += 1
                    elif WPS < risk_barrier:
                        prices[i] += np.exp(-r * (j+1)) * nominal
                            * 0.9# + accumulated_coupons * 0.1)
                        break
                elif j == 6:
                    if WPS > risk_barrier:
                        prices[i] += np.exp(-r * (j+1)) * nominal
                    else:
                        prices[i] += np.exp(-r * (j+1)) * 0.9 *
                            nominal

    return prices


answer2 = 'n'
answer2 = input("Do you want to run Task 5b? Answer y/n: ")
if answer2 =='y':
    rate_1 = 0
    rate_2 = 0
    for i in coupon_rates1:
        prices_1 = simulate_coupon_rate(paths, periods, r, sigma,
            nominal, autocall_barrier, coupon_barrier,
            risk_barrier, rho_1, i)
        price_1 = np.mean(prices_1)
        if price_1 > 99.99 and price_1 < 100.01:
            rate_1 = i
            break

    for i in coupon_rates2:
```

```
        prices_2 = simulate_coupon_rate(paths, periods, r, sigma,
            nominal, autocall_barrier, coupon_barrier,
           risk_barrier, rho_2, i)
        price_2 = np.mean(prices_2)

        if price_2 > 99.99 and price_2 < 100.01:
            rate_2 = i
            break

    print("rate_1 ",rate_1," when the price is ",price_1)
    print("rate_2 ",rate_2," when the price is ",price_2)


"""
```